

Questao 3 - Gabarito

Duracao: 1 hora

Topicos: TADs e Recursividade

Questao 1 - TAD: Data (30 pontos)

Você deve implementar um Tipo Abstrato de Dado (TAD) chamado Data, que representa uma data no formato dia/mês/ano.

a) Defina a estrutura do TAD Data, contendo os atributos e as operacoes:

- criarData(int dia, int mes, int ano)
- eBissexto()
- compararCom(Data outraData)

b) Escreva um exemplo de uso do TAD criando duas datas e comparando-as.

c) Justifique o uso de um TAD para representar datas, em vez de variaveis soltas.

Questao 2 - Recursividade: Contagem de Digitos Pares (30 pontos)

Implemente uma funcao recursiva que conte quantos digitos pares existem em um numero inteiro positivo.

a) Implemente a funcao.

b) Mostre o funcionamento da pilha de chamadas para a entrada 204.

Questao 3 - TAD: Fracao + Recursividade (40 pontos)

Implemente um TAD chamado Fracao, com as seguintes operacoes:

- criarFracao(int numerador, int denominador)
- simplificar(): reduz a fracao ao formato mais simples usando recursividade (funcao mdc).
- imprimir(): exibe a fracao simplificada.

a) Implemente o TAD com as operacoes descritas.

b) Escreva um exemplo de uso com duas frações.

Gabarito Comentado

Questao 1 - Gabarito

a)

```
class Data:
    def __init__(self, dia, mes, ano):
        self.dia = dia
        self.mes = mes
        self.ano = ano

    def eh_bissexto(self):
        return (self.ano % 4 == 0 and self.ano % 100 != 0) or (self.ano % 400 == 0)

    def comparar_com(self, outra):
        if (self.ano, self.mes, self.dia) < (outra.ano, outra.mes, outra.dia):
            return -1
        elif (self.ano, self.mes, self.dia) == (outra.ano, outra.mes, outra.dia):
            return 0
        else:
            return 1
```

b)

```
data1 = Data(5, 6, 2024)
data2 = Data(10, 6, 2023)
print(data1.comparar_com(data2)) # Saída: 1
```

c)

Um TAD melhora a organizacao e reutilizacao do codigo. Agrupar atributos e metodos evita erros e facilita o entendimento.

Questao 2 - Gabarito

a)

```
def contar_digitos_pares(n):
    if n == 0:
        return 0
    ultimo = n % 10
    return (1 if ultimo % 2 == 0 else 0) + contar_digitos_pares(n // 10)
```

b)

Para n = 204:

- contar_digitos_pares(204) -> 4 e par -> +1 -> contar_digitos_pares(20)
- contar_digitos_pares(20) -> 0 e par -> +1 -> contar_digitos_pares(2)
- contar_digitos_pares(2) -> 2 e par -> +1 -> contar_digitos_pares(0)

- contar_digitos_pares(0) -> retorna 0

Resultado: 3 pares

Questao 3 - Gabarito

a)

```
class Fracao:
    def __init__(self, numerador, denominador):
        self.numerador = numerador
        self.denominador = denominador

    def mdc(self, a, b):
        if b == 0:
            return a
        return self.mdc(b, a % b)

    def simplificar(self):
        divisor = self.mdc(self.numerador, self.denominador)
        self.numerador //= divisor
        self.denominador //= divisor

    def imprimir(self):
        print(f"{self.numerador}/{self.denominador}")
```

b)

```
f1 = Fracao(6, 8)
f1.simplificar()
f1.imprimir() # Saida: 3/4
```

```
f2 = Fracao(10, 25)
f2.simplificar()
f2.imprimir() # Saida: 2/5
```